

# Step-by-Step Details – Where and How AI Contributed

## 1. Understanding the Project Requirements

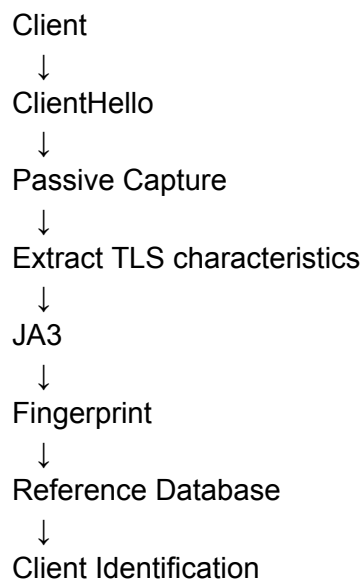
At the beginning, we first used AI to understand what the project was actually asking us to build.

We asked questions about:

- TLS fingerprinting
- ClientHello and ServerHello
- Passive packet capture
- JA3 and JA3S
- TLS cipher suites and extensions
- GREASE
- The role of Redis
- The purpose of identifying clients from fingerprints

The goal at this stage was **not to write code**, but to build enough understanding of the networking concepts before starting implementation.

For example, we first established the basic idea:



This helped us understand the complete problem before deciding how to implement it.

---

## 2. Understanding JA3 and JA3S

We then used AI to understand exactly how JA3 and JA3S are constructed.

We discussed the fields used by JA3:

TLS Version  
Cipher Suites  
Extensions  
Elliptic Curves / Supported Groups  
EC Point Formats

and how these fields are converted into a JA3 string and subsequently into an MD5 fingerprint.

For example:

TLS fields  
↓  
JA3 string  
↓  
MD5  
↓  
32-character fingerprint

We also used AI to understand JA3S and the difference between the client-side and server-side fingerprints.

---

## 3. Understanding GREASE

One of the concepts that required additional clarification was **GREASE**.

We used AI to understand:

- What GREASE values are.
- Why do modern TLS implementations use them?
- Why do they appear in cipher and extension lists?
- Why JA3 removes them before constructing the fingerprint.

This helped us avoid treating GREASE as ordinary fingerprint data.

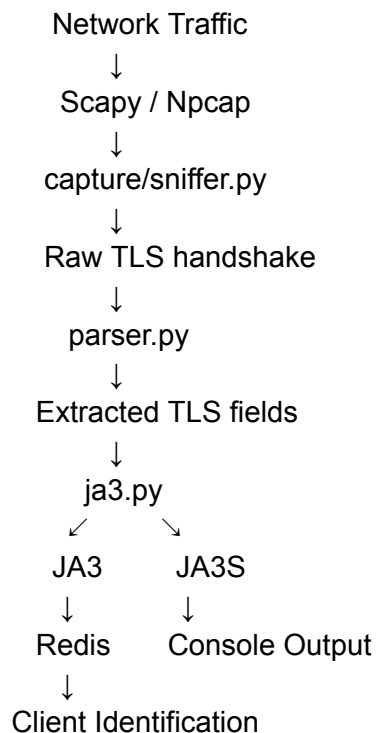
The official JA3 implementation explicitly filters GREASE values so that clients using GREASE can still produce a consistent JA3 fingerprint.

---

## 4. Designing the High-Level Architecture

After understanding the concepts, we designed the architecture ourselves and used AI to **review and discuss** the proposed structure.

The resulting architecture was:



We decided to keep the components separate so that packet capture, parsing, fingerprint generation, storage, and identification could be understood and tested independently.

AI was used to review whether the proposed architecture was reasonable for the course project and whether any major requirement had been missed.

The **final architecture and technology choices were ours**.

---

## 5. Implementing the TLS Parser

The main area where we used AI for direct implementation assistance was the **TLS parser**.

We wanted to understand the raw TLS structure instead of depending entirely on a high-level TLS library.

We first asked AI to explain the byte-level structure of ClientHello and ServerHello:

TLS Record  
↓  
Handshake Header  
↓  
ClientHello / ServerHello  
↓  
Version  
Random  
Session ID  
Cipher information  
Compression  
Extensions

We then used that understanding to implement `parser.py`.

AI assistance was mainly useful for:

- Understanding byte offsets.
- Understanding length-prefixed fields.
- Extracting cipher suites.
- Parsing extension blocks.
- Extracting supported groups.
- Extracting EC point formats.
- Understanding ServerHello structure.
- Checking edge cases and malformed/incomplete data.

The parser was then integrated and tested by us against actual captured TLS traffic.

The final implementation was not treated as correct simply because AI suggested it. We verified its output using our own packet captures and subsequent JA3 validation.

---

## 6. Implementing JA3 and JA3S

The JA3/JA3S construction itself was implemented according to the specification.

AI was primarily used to:

- Clarify the required field ordering.
- Check our implementation.
- Explain GREASE filtering.
- Help reason about MD5 hashing.
- Identify potential formatting mistakes.

For example, we checked that the JA3 string followed:

Version,  
Ciphers,  
Extensions,  
Curves,  
Point Formats

and that the resulting string was hashed using MD5. This agrees with the published JA3 specification and reference implementation.

The important distinction was that AI helped us **verify and understand the implementation**, while we remained responsible for integrating it into our project.

---

## 7. Redis Database and Fingerprint Registration

We decided to use Redis as the reference fingerprint database.

The basic idea was:

JA3 hash  
↓  
Redis  
↓  
Client name

For example:

JA3 hash → curl  
JA3 hash → Python  
JA3 hash → Node  
JA3 hash → OpenSSL  
JA3 hash → Custom Client

AI was used to discuss how the database could be structured and how `populate_db.py` and `live_identify.py` should interact with it.

There was also some implementation assistance while developing the fingerprint registration functionality.

However, the actual fingerprints we registered, the clients we selected, and the experimental process used to generate them were decided and performed by us.

---

## 8. Fuzzy Matching

We added fuzzy matching as an additional feature rather than making it a core requirement.

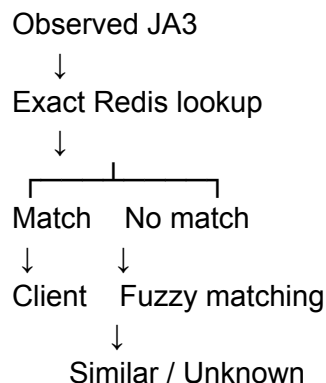
The purpose was to handle situations where an observed JA3 did not exactly match a fingerprint in Redis but appeared structurally similar to a known fingerprint.

AI helped us reason about:

- Comparing JA3 fields.
- Similarity between cipher/extension/curve sets.
- Choosing a similarity threshold.
- Handling exact-match failures.

We implemented a weighted similarity approach as a fallback after exact Redis lookup.

The workflow became:



During testing, fuzzy matching for curl sometimes failed because small differences in the observed TLS parameters could lower the similarity score below the selected threshold.

This also helped us understand an important limitation of fingerprint matching: **similarity-based identification is a heuristic and should not be treated as guaranteed identity.**

---

## 9. Testing with Real Network Traffic

We then moved from individual components to the complete live system.

We used Scapy/Npcap to capture real traffic from the selected network interface.

The testing workflow was:

Generate HTTPS request



Network interface



Npcap



Scapy



TLS ClientHello



parser.py



JA3



Redis lookup



Identified client

We tested multiple clients/tools, including:

- curl
- Python
- Node.js
- OpenSSL
- Custom TLS client

AI was mainly used during this stage for troubleshooting commands, interpreting errors, and suggesting what to check when the expected ClientHello was not appearing.

The actual traffic generation and packet capture experiments were performed by us.

---

## 10. Debugging Unknown Fingerprints

One issue we encountered was that the live sniffer sometimes produced:

JA3 → unknown

even when we were testing a known client.

We used AI to investigate possible reasons.

We identified that the sniffer was operating passively on the network interface, so it could see HTTPS traffic generated by **other applications running on the machine as well**.

Therefore:

Our test client

+

Background applications

+

Browser traffic

+

Other HTTPS services

↓

Same network interface

↓

Scapy sees all of them

This explained why unknown fingerprints could appear.

We also discussed the limitations of registering the first ClientHello seen by `populate_db.py` and the importance of controlling the test environment during fingerprint registration.

---

## 11. Windows / Scapy / Npcap Issues

Because the project was developed on Windows, some issues were related to the environment rather than the fingerprinting algorithm.

AI was used to understand:

- What `iface` means in Scapy.
- How Npcap provides packet capture on Windows.
- How to identify the correct Wi-Fi interface.
- Why Docker/Redis sometimes could not be started.
- Why Linux-style `/dev/null` commands behave differently in PowerShell.
- How to generate HTTPS traffic correctly using `curl.exe`.

These were mainly **debugging and environment-support tasks**, rather than project design decisions.

---

## 12. Validation Against Published JA3 Data

After the implementation was working, we wanted to verify that our output was not simply an internally consistent hash.

We therefore used a published JA3 example.

The published example uses:

769,47-53-5-10-49161-49162-49171-49172-50-56-19-4,0-10-11,23-24-25,0

with the expected fingerprint:

ada70206e40642a3e4461f35503241d5

Our implementation reproduced the same fingerprint.

This was an important validation step because it checked our JA3 string construction and hashing against an external reference rather than only against our own code. The same canonical example is documented by Salesforce.

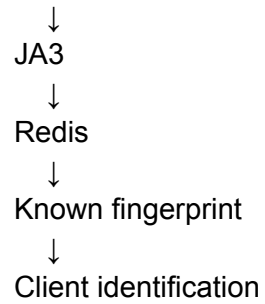
---

## 13. Final Multi-Client Demonstration

Finally, we used the completed system to demonstrate that different TLS clients could produce distinguishable fingerprints.

Our reference database contained fingerprints for multiple tools, allowing the system to perform:

ClientHello



We also examined unknown fingerprints and fuzzy matches instead of assuming that every observed fingerprint would always have an exact database entry.

The final browser/curl/custom-client comparison was used to demonstrate the main purpose of the project: identifying TLS client behavior from the handshake without inspecting encrypted application data. This is consistent with the motivation behind JA3, where ClientHello characteristics can be used to fingerprint client applications.